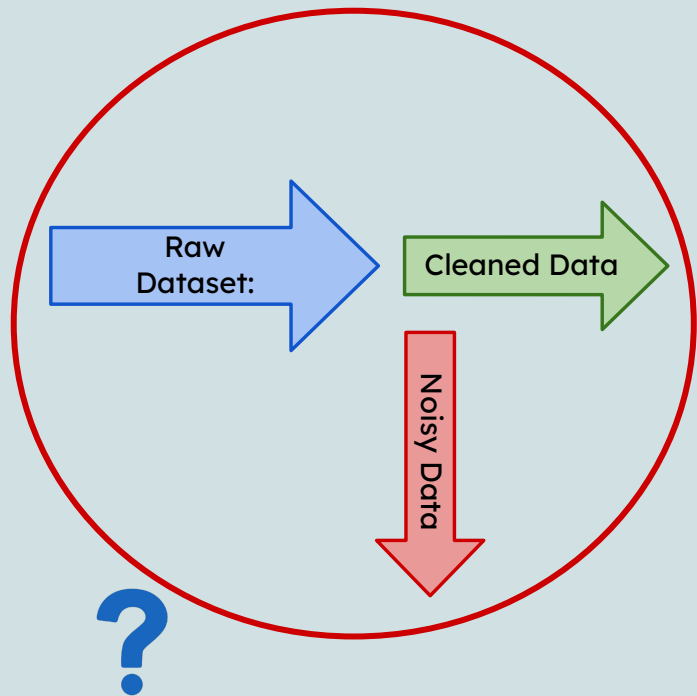


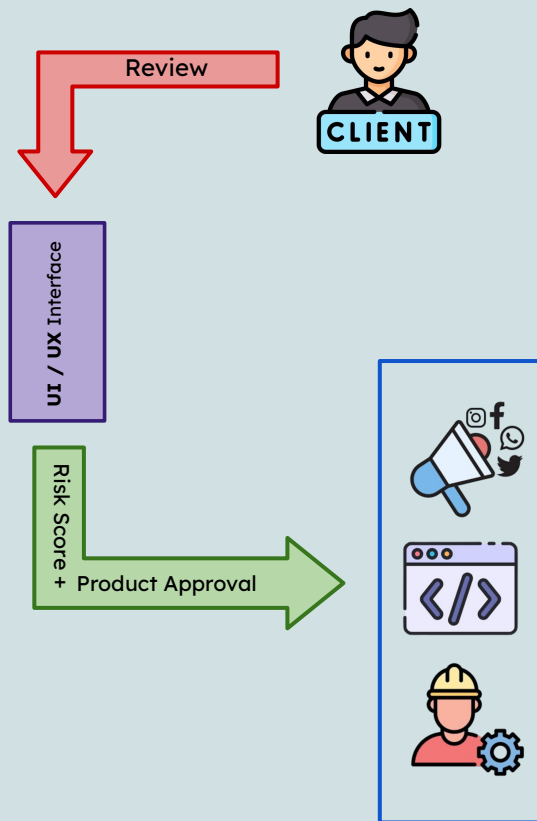
Developer Side



AI / ML or AI-Agent Model

Save .pkl file with Trained Model

User Side



Part a: How Do We Separate Noisy Data vs Useful Data

Step 1:

Identify and Remove Incomplete Data

```
=== Missing Value Statistics ===  
Missing Count Missing Percent
```

```
subtitle 4132 100.00  
author 4132 100.00  
bought together 4132 100.00
```

```
price 1428 34.56  
color 284 6.87
```

```
main_category 28 0.68  
os 15 0.36
```

```
rating 0 0.00
```

```
helpful_vote 0 0.00
```

```
timestamp 0 0.00
```

```
user_id 0 0.00
```

```
asin 0 0.00
```

```
title_x 0 0.00
```

```
text 0 0.00
```

```
title_y 0 0.00
```

```
average_rating 0 0.00
```

```
categories 0 0.00
```

```
store 0 0.00
```

```
rating_number 0 0.00
```

```
features 0 0.00
```

```
avg_helpful_votes 0 0.00
```

```
num_reviews 0 0.00
```

```
id 0 0.00
```

```
brand 0 0.00
```

Does not matter, as those weren't provided

Not Essential for Risk Analysis + Too Much Data Loss if Removed

"Title_x" and "rating" are 100%, so these rows can be kept

No Data Lost

Part a: How Do We Separate Noisy Data vs Useful Data

Step 2:

Determining a Statistically Stable Training Period



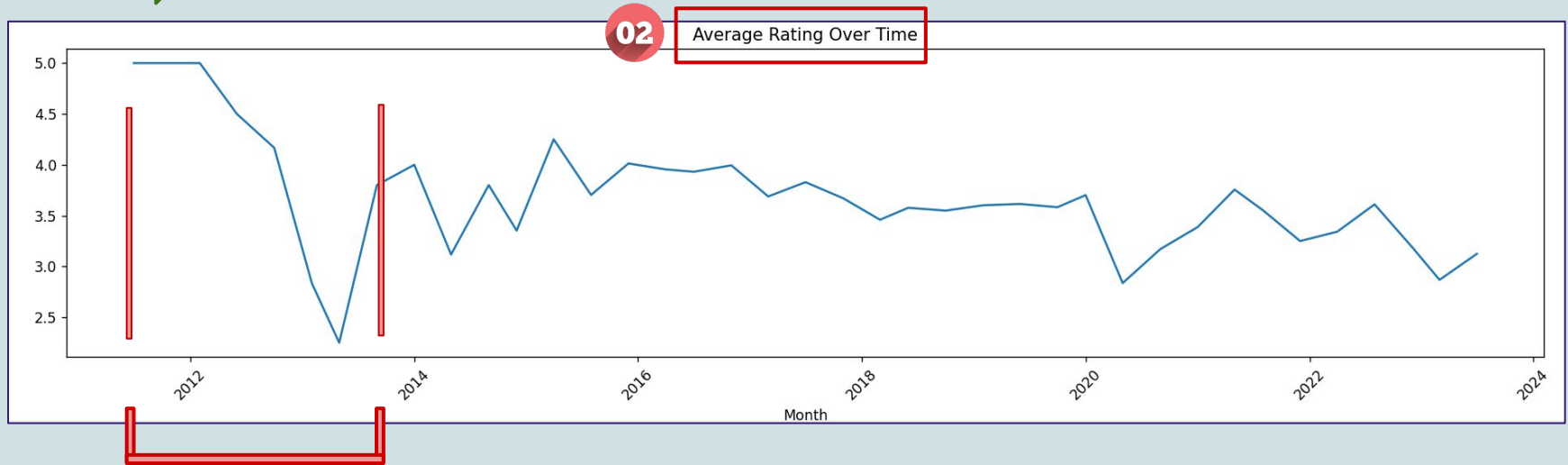
Training on early & low-volume years creates a **mis-specified baseline mean**, teaching the model that minimal engagement is “normal” for a **\$2.8B company**.

As a result, a real decline in reviews volume, engagement, or complain signals may not trigger alerts, which is the entire point of this model.

Part a: How Do We Separate Noisy Data vs Useful Data

Step 2:

Determining a Statistically Stable Training Period



Once again, **prior to 2014**, the early low-volume years exhibit a **rapid** and **unpredictable change** in **average ratings**.

This would inflate the **baseline variance** and **reduce the model's sensitivity** to genuine rating volatility in the future.

Part a: How Do We Separate Noisy Data vs Useful Data

Step 2:

Determining a Statistically Stable Training Period



It is very unlikely a company with **\$2.8B** in annual revenue will have <10 unique product reviews per month

Like previous scenarios, **incorporating this subset would only increase insensitivity to unusual circumstances.**

Part a: How Do We Separate Noisy Data vs Useful Data

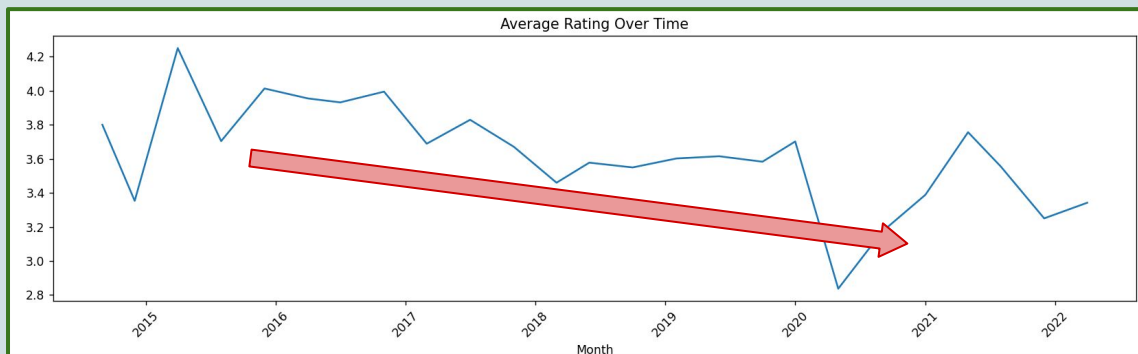
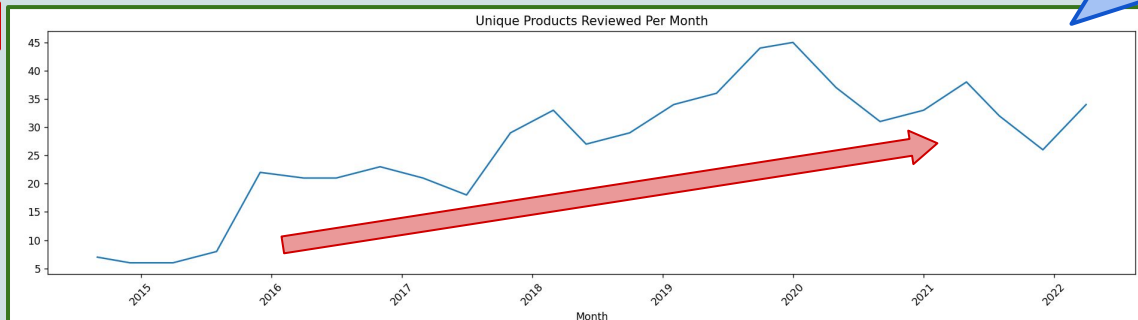
Step 3:

Cleaning the Dataset (Visual Overview)

```
start_date = "2014-06-01"  
end_date = "2022-06-30"
```

```
Original rows: 4132  
Filtered rows: 3906  
Rows removed: 226  
Percentage removed: 5.47%  
Filtered dataset saved.
```

Small, Controlled Reduction



Part a: How Do We Separate Noisy Data vs Useful Data

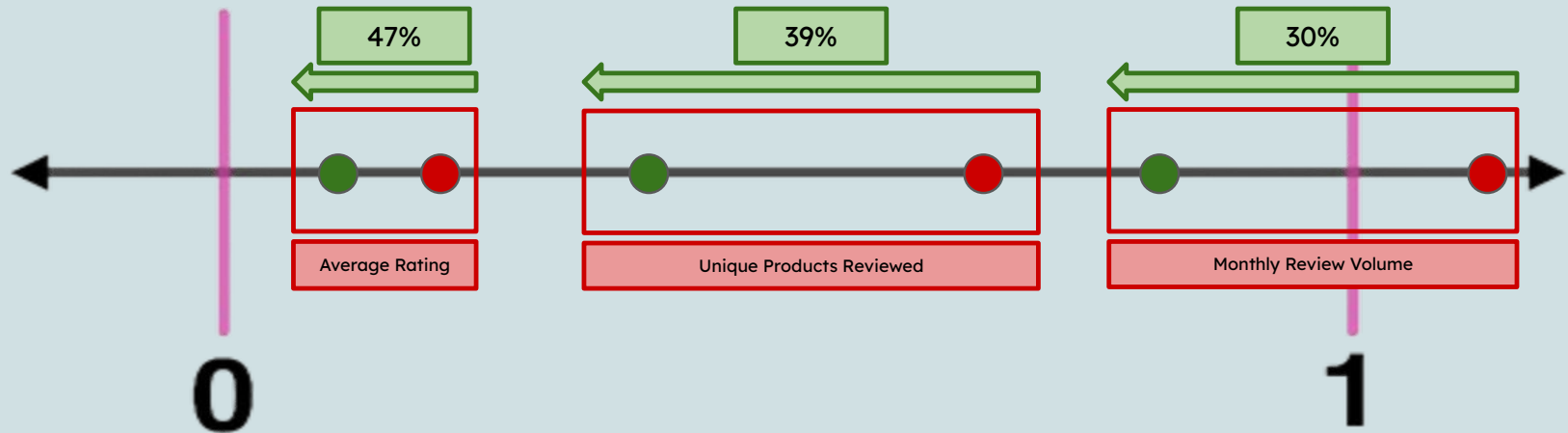
Step 3:

Cleaning the Dataset (Quantitative Overview)

Coefficient of Variation (CV): standard deviation / mean

Essentially, how **volatile** or **unpredictable** is the dataset?

Lower is better for
training models



Part a: How Do We Separate Noisy Data vs Useful Data

Step 4:

Deleting the Duplicates + 3 Empty Columns

Checks if a review has been accidentally duplicated by the system

```
----- SAME TEXT + SAME RATING + ID Difference ≤ 1 -----
```

Total duplicate clusters found: 34

--- Example Group 1 ---

ID: 23569 | Rating: 5 | Text: Already love it, my only issue is the screen, but it isn't that bad, also they d

ID: 23570 | Rating: 5 | Text: Already love it, my only issue is the screen, but it isn't that bad, also they d

--- Example Group 2 ---

ID: 22422 | Rating: 5 | Text: Arrived (to Canada) a week before scheduled delivery.
Bright screen and fas

ID: 22423 | Rating: 5 | Text: Arrived (to Canada) a week before scheduled delivery.
Bright screen and fas

--- Example Group 3 ---

ID: 6664 | Rating: 5 | Text: Battery life is not the best and its a very large laptop so I wouldnt recommend

ID: 6665 | Rating: 5 | Text: Battery life is not the best and its a very large laptop so I wouldnt recommend

... and 31 more groups not shown.

Rows removed from proximity duplicates: 37

New dataset size: 3869

Original columns: 25

Columns with no usable values:

- bought_together
- subtitle
- author

Columns after removal: 22
Final cleaned dataset saved.

Part a: How Do We Separate Noisy Data vs Useful Data

Summary

FusionTech Online Reviews.csv



FusionTech_2014_06_to_2022_06_final.csv

Time Period Resizing

226 Rows

Duplicate Rows Deletion

37 Rows

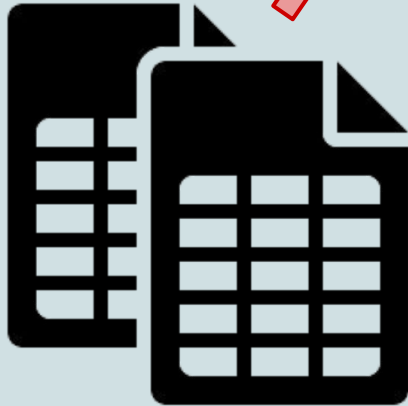
$(263 / 4132) * 100 = 6.37\%$ Data Reduction in Cleaning

Quick Detour: Where should we store these large datasets & how to be energy efficient with large datasets?

Option #1

No Live Updates + Multiple Users Cannot Use + Not Professional

Local Machine



.csv Review Files

Uses Local Memory + Energy Inefficient + Slow at Indexing Large Data



Developer Code

Quick Detour: Where should we store these large datasets & how to be energy efficient with large datasets?

Option #2

Structured, Relational Data + Scalable for Real Infrastructure

Global Server



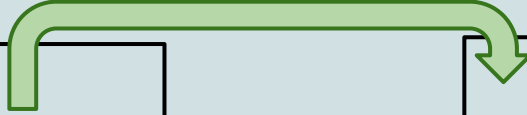
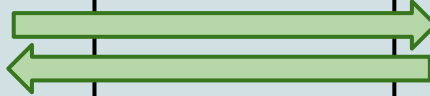
Dataset Stored in Neon SQL

Local Machine

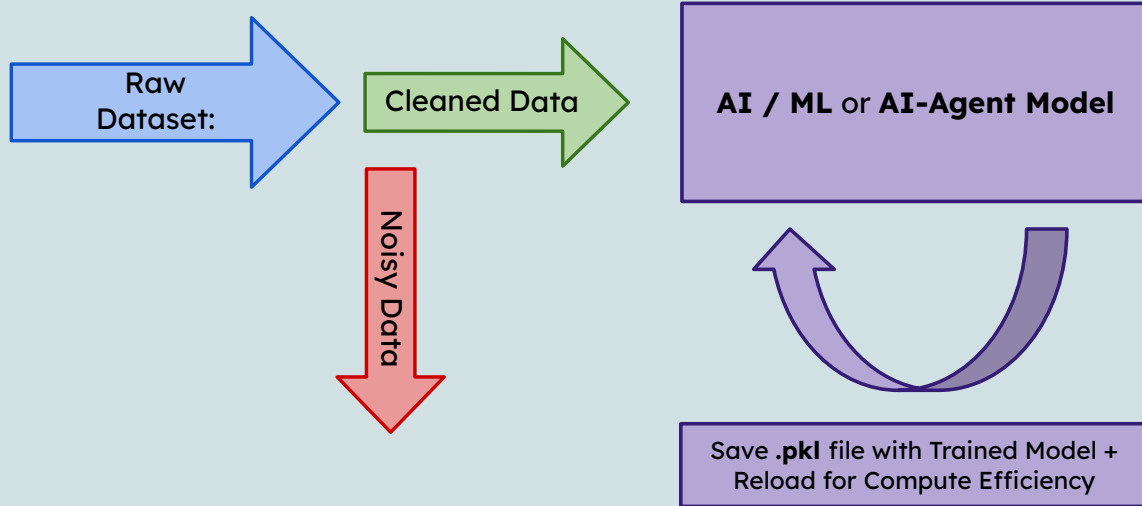


Developer Code

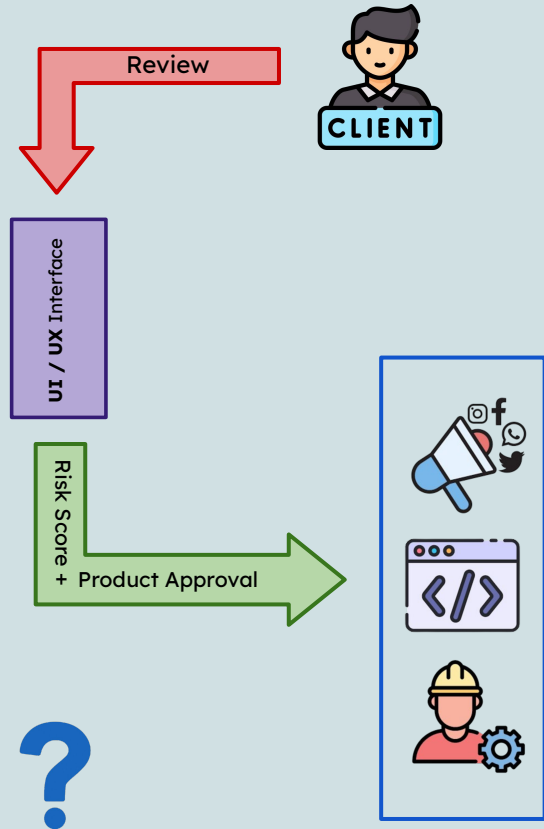
Optimal Performance & Speed on Large Data



Developer Side



User Side



Part b: How Can We Validate our Model?

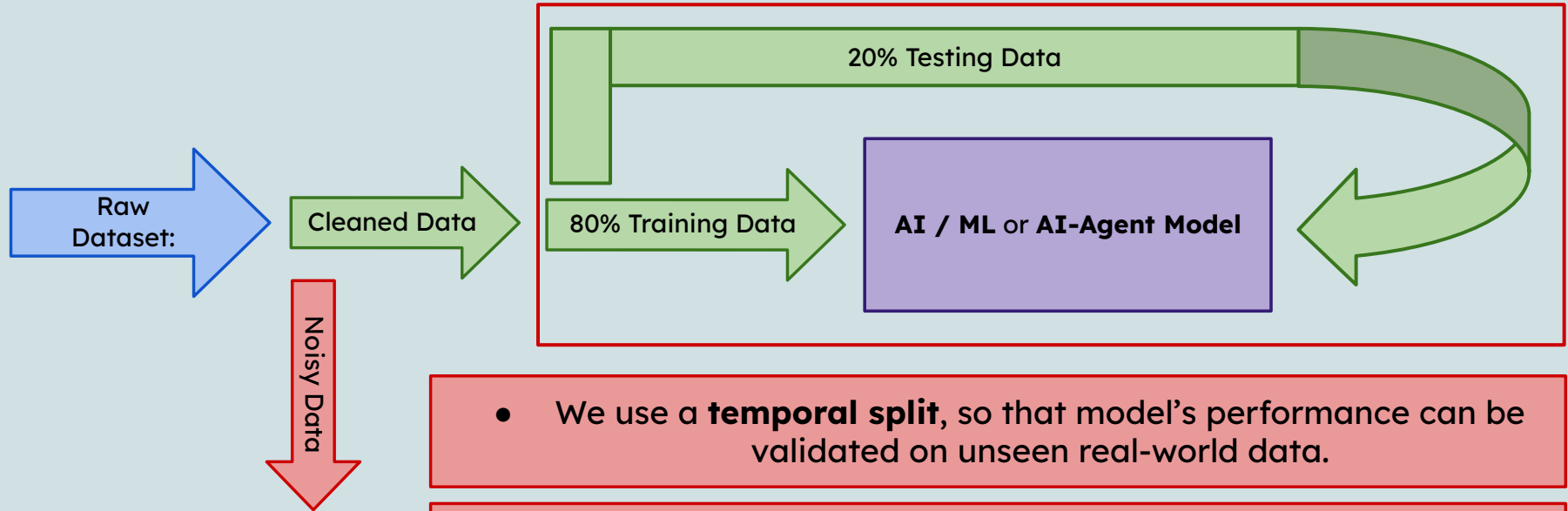
Option 1 - 80/20 Temporal Split of Given Dataset

Option 2 - Generating Synthetic Support Tickets

Option 3 - Historical Revenue Impact Replay

Part b: How Can We Validate our Model?

Option 1 - 80/20 Temporal Split of Given Dataset

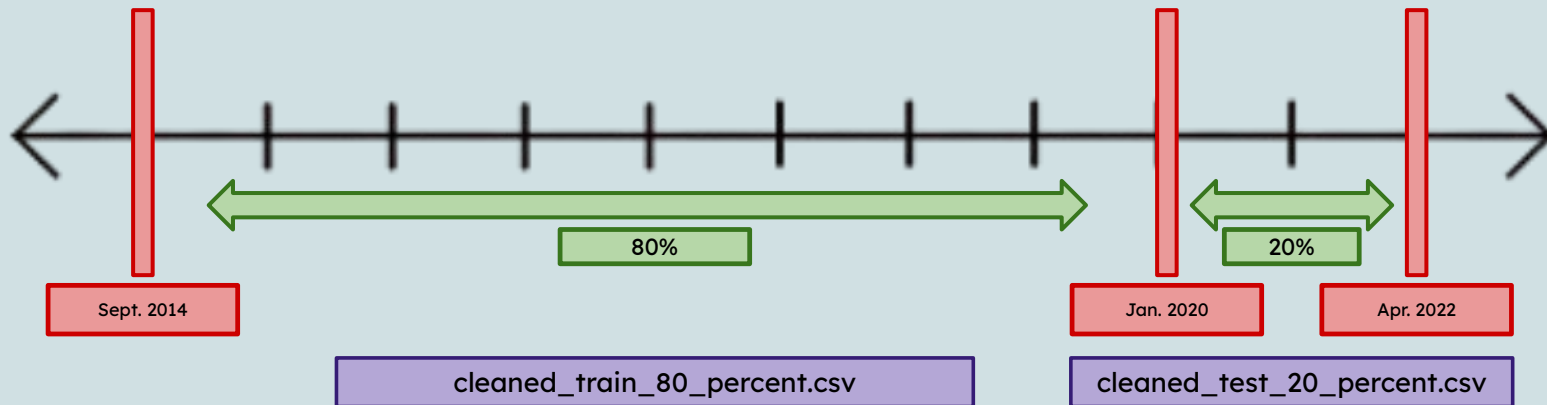


- We use a **temporal split**, so that model's performance can be validated on unseen real-world data.
- Synthetic data we create cannot fully replicate the **complexity** and **unpredictability** of **authentic customer behavior**.

Part b: How Can We Validate our Model?

Option 1 - 80/20 Temporal Split of Given Dataset

- A random 90/10 split would mix past and future data, introducing look-ahead bias.



- Since we are modeling time-dependent risk, we sort the data chronologically.
- The earlier 80% is used for training, and the most recent 20% is used for testing.

Part b: How Can We Validate our Model?

Option 1 - 80/20 Temporal Split of Given Dataset - Results

```
Total Reviews: 3869
```

```
Training Reviews (80%): 3095
```

```
Testing Reviews (20%): 774
```



Dataset Split into **Training & Testing**

```
--- Training Period ---
```

```
Start: 2014-09-06
```

```
End: 2020-01-26
```



Training Dataset spans ~6 years

```
--- Testing Period ---
```

```
Start: 2020-01-26
```

```
End: 2022-04-15
```



Testing Dataset spans ~2 years

```
Training dataset saved.
```

```
Testing dataset saved.
```


Step 3: How Can We Validate Our Model?

Option 2 - Generating Synthetic Support Tickets

v1 - No Trend

Start - Synthetic Support Tickets + Reviews



Stop - Synthetic Support Tickets + Reviews

Pass Over Span on Months

**Trained
AI / ML or AI-Agent Model**

Visualization of Product Risk

Goal: Report that there is no “new” risk based on the inputs



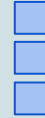
Note: This is an example of what output should look like

Step 3: How Can We Evaluate Our Model?

Option 2 - Generating Synthetic Support Tickets

v2 - Small Trend, Long-Term Impact

Start - Synthetic Support Tickets + Reviews



Stop - Synthetic Support Tickets + Reviews

Pass Over Span on Months

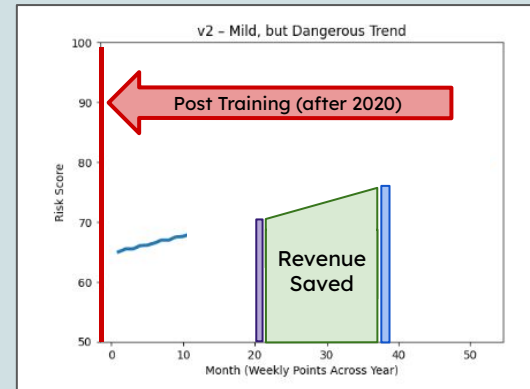
**Trained
AI / ML or AI-Agent Model**

Visualization of Product Risk

Goals:

1) Determine when the trend would have been caught manually through analyzing the dataset, and come up with a reasonable 'delay'.

2) Evaluate when a risk was flagged by the model, and how much revenue was 'saved.'



Note: This is an example of what output should look like

Step 3: How Can We Evaluate Our Model?

This is the **most** unique part of our project!

Option 3 - Historical Revenue Impact Replay

Goal: If our system existed in 2016, it would have triggered an alert in Month X, 3 months before average rating dropped, protecting an estimated \$Y in revenue.



Revenue Damage Control $\approx (\$2.8\text{B} / 12 \text{ months}) \times (1/40 \text{ products}) \times 4 \text{ months} \times (30\% \text{ to } 50\% \text{ recovery}) \approx \text{\$7 to \$12 Million}$

Next Step:

Designing our Automated Model

Col. 1
Col. 2
Col. 3
○
○
○
Col. n

Input: Cleaned Data
via **Neon SQL**

AI / ML or AI-Agent Model

Save **.pkl** file with Trained Model +
Reload for Compute Efficiency

Output:

Risk Analysis for
Each Product

Chatbot Summary of
Product Progression

Model Validation

